```
"""
StegSim config.py — Simulation configuration loader and validator.
All parameters are configurable. Defaults match v1 VBDS spec.
"""

import json
from dataclasses import dataclass, field
from pathlib import Path
from typing import Optional


@dataclass
class DriftConfig:
    trust_decay_per_step:             float = 0.001
    authority_staleness_per_step:     float = 0.002
    resource_concentration_per_step:  float = 0.0015
    policy_lag_growth_per_step:       float = 0.002
    mutation_pressure_growth_per_step: float = 0.001
    fragmentation_growth_per_step:    float = 0.0008
    communication_decay_per_step:     float = 0.0005
    shock_probability:                float = 0.02
    shock_magnitude:                  float = 0.15


@dataclass
class InitialConditions:
    trust_mean:             float = 0.72
    trust_std:              float = 0.10
    authority_mean:         float = 0.65
    authority_std:          float = 0.10
    resource_balance_mean:  float = 0.68
    resource_balance_std:   float = 0.12
    alignment_mean:         float = 0.74
    alignment_std:          float = 0.08
    autonomy_mean:          float = 0.50
    autonomy_std:           float = 0.10
    policy_freshness:       float = 1.0
    neighbor_count_mean:    int   = 5


@dataclass
class Thresholds:
    healthy:      float = 0.70
    degraded:     float = 0.45
```

```
    warning:      float = 0.25
    inadmissible: float = 0.25
    # Legitimacy capacity params
    legitimacy_k:      float = 1.0
    legitimacy_alpha:  float = 0.4
    legitimacy_beta:   float = 0.3
    legitimacy_gamma:  float = 0.3
    # Authority staleness
    max_acceptable_policy_lag: int = 10
    # Autonomy excess threshold
    autonomy_excess_factor: float = 1.05


@dataclass
class WeightsConfig:
    trust_continuity:   float = 0.25
    authority_coherence: float = 0.25
    resource_balance:   float = 0.15
    policy_freshness:   float = 0.15
    mutation_pressure:  float = 0.10
    fragmentation:      float = 0.10


@dataclass
class SimConfig:
    scenario:           str   = "stale_authority_drift"
    run_id:             str   = "demo-stale-authority-001"
    seed:               int   = 1729
    agents:             int   = 10000
    steps:              int   = 250
    output_dir:         str   = "runs"
    report_every:       int   = 10     # emit metrics every N steps
    initial:            InitialConditions = field(default_factory=InitialConditio
    drift:              DriftConfig       = field(default_factory=DriftConfig)
    thresholds:         Thresholds        = field(default_factory=Thresholds)
    weights:            WeightsConfig     = field(default_factory=WeightsConfig)
    # v2+ fields (ignored in v1, present for forward compatibility)
    enable_hybrid_agents:      bool  = False
    hybrid_agent_fraction:     float = 0.0
    enable_topology_mutation:  bool  = False
    enable_reconstruction:     bool  = False
    enable_phase_diagram:      bool  = False
    phase_diagram_runs:        int   = 0

    @classmethod
    def from_file(cls, path: str) -> "SimConfig":
        with open(path) as f:
```

```
            data = json.load(f)
        return cls.from_dict(data)

    @classmethod
    def from_dict(cls, data: dict) -> "SimConfig":
        cfg = cls()
        cfg.scenario  = data.get("scenario",  cfg.scenario)
        cfg.run_id    = data.get("run_id",    cfg.run_id)
        cfg.seed      = data.get("seed",      cfg.seed)
        cfg.agents    = data.get("agents",    cfg.agents)
        cfg.steps     = data.get("steps",     cfg.steps)
        cfg.output_dir = data.get("output_dir", cfg.output_dir)
        cfg.report_every = data.get("report_every", cfg.report_every)

        if "initial_conditions" in data:
            ic = data["initial_conditions"]
            cfg.initial = InitialConditions(**{
                k: ic[k] for k in ic if hasattr(cfg.initial, k)
            })

        if "drift" in data:
            d = data["drift"]
            cfg.drift = DriftConfig(**{
                k: d[k] for k in d if hasattr(cfg.drift, k)
            })

        if "thresholds" in data:
            t = data["thresholds"]
            cfg.thresholds = Thresholds(**{
                k: t[k] for k in t if hasattr(cfg.thresholds, k)
            })

        if "weights" in data:
            w = data["weights"]
            cfg.weights = WeightsConfig(**{
                k: w[k] for k in w if hasattr(cfg.weights, k)
            })

        # v2+ optional fields
        cfg.enable_hybrid_agents     = data.get("enable_hybrid_agents",     False
        cfg.hybrid_agent_fraction    = data.get("hybrid_agent_fraction",    0.0)
        cfg.enable_topology_mutation = data.get("enable_topology_mutation", False
        cfg.enable_reconstruction    = data.get("enable_reconstruction",    False
        cfg.enable_phase_diagram     = data.get("enable_phase_diagram",     False
        cfg.phase_diagram_runs       = data.get("phase_diagram_runs",       0)

        return cfg
```

```
def to_dict(self) -> dict:
    import dataclasses
    return dataclasses.asdict(self)

def weights_dict(self) -> dict:
    import dataclasses
    return dataclasses.asdict(self.weights)
```